

Eigenstate formatting when using VQE with GPU

<https://github.com/Qiskit/qiskit-aer/issues/1539>

ROW 15

Eigenstate formatting when using VQE with GPU #1539

Closed



Red2Je opened on Jun 14, 2022 · edited by Red2Je

Edits ...

Informations

- Qiskit Aer version:
- 0.10.4
- Python version:
- 3.8.10
- Operating system:
- Windows 10 using WSL2 5.10.16

What is the current behavior?

Related to the message from this open user : <https://qiskit.slack.com/archives/C88B13092/p1655043788534019> , using AerSimulator with VQE and the GPU option enabled returns the eigenstate in the wrong format.

Steps to reproduce the problem

```
from qiskit import Aer
from qiskit.providers.aer import AerSimulator
from qiskit.opflow import X, Z, I
from qiskit.utils import QuantumInstance, algorithm_globals
from qiskit.algorithms import VQE
from qiskit.algorithms.optimizers import SLSQP
from qiskit.circuit.library import TwoLocal

H2_op = (-1.052373245772859 * I ^ I) + \
        (0.39793742484318045 * I ^ Z) + \
        (-0.39793742484318045 * Z ^ I) + \
        (-0.01128010425623538 * Z ^ Z) + \
        (0.18093119978423156 * X ^ X)

seed = 50
algorithm_globals.random_seed = seed
backend_cpu = AerSimulator(method = 'statevector')
backend_gpu = AerSimulator(method = 'statevector', device = 'GPU')
qi_cpu = QuantumInstance(backend_cpu, seed_transpiler=seed, seed_simulator=seed)
qi_gpu = QuantumInstance(backend_gpu, seed_transpiler=seed, seed_simulator=seed)

ansatz = TwoLocal(rotation_blocks='ry', entanglement_blocks='cz')
optimizer = SLSQP(maxiter=1000)
vqe_cpu = VQE(ansatz, optimizer=optimizer, quantum_instance=qi_cpu)
vqe_gpu = VQE(ansatz, optimizer=optimizer, quantum_instance=qi_gpu)
result_cpu = vqe_cpu.compute_minimum_eigenvalue(operator=H2_op)
result_gpu = vqe_gpu.compute_minimum_eigenvalue(operator=H2_op)

print("CPU : \n" , result_cpu)
print("GPU : \n", result_gpu)
```

Output :

```
CPU :
{ 'aux_operator_eigenvalues': None,
  'cost_function_evals': 65,
  'eigenstate': array([ 9.55146158e-05+0.j, -9.93766272e-01+0.j,  1.11483575e-01+0.j,
                        1.77493990e-05+0.j]),
  'eigenvalue': (-1.8572750175664259+0j),
  'optimal_parameters': { ParameterVectorElement(0[4]): -2.5983258978150006,
                        ParameterVectorElement(0[5]): 1.5683260003556614,
                        ParameterVectorElement(0[6]): -4.717618171283927,
                        ParameterVectorElement(0[7]): 0.3602072577510429,
                        ParameterVectorElement(0[0]): 4.296520455019831,
                        ParameterVectorElement(0[1]): 4.426962083985579,
                        ParameterVectorElement(0[2]): 0.5470754664946292,
                        ParameterVectorElement(0[3]): 6.092947779034692},
  'optimal_point': array([ 4.29652046,  4.42696208,  0.54707547,  6.09294778, -2.5983259,
```

```

ParameterVectorElement(0[7]): 0.3602072577510429,
ParameterVectorElement(0[0]): 4.296520455019831,
ParameterVectorElement(0[1]): 4.426962083985579,
ParameterVectorElement(0[2]): 0.5470754664946292,
ParameterVectorElement(0[3]): 6.092947779034692},
'optimal_point': array([ 4.29652046,  4.42696208,  0.54707547,  6.09294778, -2.5983259 ,
 1.568326 , -4.71761817,  0.36020726]),
'optimal_value': -1.8572750175664259,
'optimizer_evals': None,
'optimizer_time': 0.3249850273132324}
GPU :
{ 'aux_operator_eigenvalues': None,
  'cost_function_evals': 9,
  'eigenstate': { '01': 0.6123724356957945,
                  '10': 0.20009763241977652,
                  '11': 0.764827390657526},
  'eigenvalue': (-1.3670071702848867+0j),
  'optimal_parameters': { ParameterVectorElement(0[4]): -2.1556830785333663,
                          ParameterVectorElement(0[5]): -4.710416180758196,
                          ParameterVectorElement(0[6]): -4.882565275402284,
                          ParameterVectorElement(0[7]): 6.08932895996748,
                          ParameterVectorElement(0[0]): -1.227012055676525,
                          ParameterVectorElement(0[1]): -2.940960526404766,
                          ParameterVectorElement(0[2]): 6.115830155139623,
                          ParameterVectorElement(0[3]): -1.2903442440134034},
  'optimal_point': array([-1.22701206, -2.94096053,  6.11583016, -1.29034424, -2.15568308,
 -4.71041618, -4.88256528,  6.08932896]),
  'optimal_value': -1.3670071702848867,
  'optimizer_evals': None,
  'optimizer_time': 0.6060364246368408}

```

What is the expected behavior?

The expected output would be :

```

CPU :
{ 'aux_operator_eigenvalues': None,
  'cost_function_evals': 65,
  'eigenstate': array([ 9.55146158e-05+0.j, -9.93766272e-01+0.j,  1.11483575e-01+0.j,
 1.77493990e-05+0.j]),
  'eigenvalue': (-1.8572750175664259+0j),
  'optimal_parameters': { ParameterVectorElement(0[4]): -2.5983258978150006,
                          ParameterVectorElement(0[5]): 1.5683260003556614,
                          ParameterVectorElement(0[6]): -4.717618171283927,
                          ParameterVectorElement(0[7]): 0.3602072577510429,
                          ParameterVectorElement(0[0]): 4.296520455019831,
                          ParameterVectorElement(0[1]): 4.426962083985579,
                          ParameterVectorElement(0[2]): 0.5470754664946292,
                          ParameterVectorElement(0[3]): 6.092947779034692},
  'optimal_point': array([ 4.29652046,  4.42696208,  0.54707547,  6.09294778, -2.5983259 ,
 1.568326 , -4.71761817,  0.36020726]),
  'optimal_value': -1.8572750175664259,
  'optimizer_evals': None,
  'optimizer_time': 0.3249850273132324}
GPU :
{ 'aux_operator_eigenvalues': None,
  'cost_function_evals': 9,
  'eigenstate': array([ 9.55146158e-05+0.j, -9.93766272e-01+0.j,  1.11483575e-01+0.j,
 1.77493990e-05+0.j]),
  'eigenvalue': (-1.3670071702848867+0j),
  'optimal_parameters': { ParameterVectorElement(0[4]): -2.1556830785333663,
                          ParameterVectorElement(0[5]): -4.710416180758196,
                          ParameterVectorElement(0[6]): -4.882565275402284,
                          ParameterVectorElement(0[7]): 6.08932895996748,
                          ParameterVectorElement(0[0]): -1.227012055676525,
                          ParameterVectorElement(0[1]): -2.940960526404766,
                          ParameterVectorElement(0[2]): 6.115830155139623,
                          ParameterVectorElement(0[3]): -1.2903442440134034},
  'optimal_point': array([-1.22701206, -2.94096053,  6.11583016, -1.29034424, -2.15568308,
 -4.71041618, -4.88256528,  6.08932896]),
  'optimal_value': -1.3670071702848867,
  'optimizer_evals': None,
  'optimizer_time': 0.6060364246368408}

```

Suggested solutions



optimizer_time : 0.00003642463684087

Suggested solutions



Red2Je added **bug** on Jun 14, 2022



grossiM on Jun 14, 2022

...

Hi, I posted the question, let me know if I can give you additional details.



doichanj self-assigned this on Jun 15, 2022



hhorii on Jun 16, 2022

Collaborator ...

CPU uses `AerPauliExpectation()`, which provides better performance but GPU does not because `is_aer_qasm()` checks backend name is `aer_simulator_statevector` though the backend name of GPU is `aer_simulator_statevector_gpu`.

The easiest workaround is to set `expectation` argument with `AerPauliExpectation`. We believe that this is an issue in terra, but may change the naming rule of backends in the next release.



ikkoham on Jun 17, 2022

Collaborator ...

I think `is_statevector_backend` function in Terra is problematic, but my PR was closed: [Qiskit/qiskit#7015](#). The issue of identifying the type of simulator needs to be discussed more.

I'm glad to see that workaround.



hhorii on Nov 1, 2022

Collaborator ...

Please let me close this issue because this is an issue in terra ([Qiskit/qiskit#7010](#)). I believe that this issue can be worked around by setting `expectation` in VQE construction.



hhorii closed this as **completed** on Nov 1, 2022



hhorii on Nov 1, 2022

Collaborator ...

FYI: [Qiskit/qiskit#8411](#) fixes 7010.

